

EXPERIMENT 1: INHERITANCE & METHOD OVERRIDING (ANIMAL KINGDOM)

1. Problem Statement / Question

- Write a Java program demonstrating **inheritance**, **data encapsulation**, and **runtime polymorphism (method overriding)**:
- Design a base class `Animal` with private attributes: `name`, `habitat`, and `food`. Implement a parameterized constructor, public getter methods, a base method `sound()` returning generic output, and a `show()` method printing animal properties.
 - Derive three specialized subclasses: `cow`, `pig`, and `horse`. Each subclass passes specialized initial details via `super()` and overrides the `sound()` method to produce species-specific sounds ("moo", "oink", "neigh").
 - In driver class `LabProgram3`, instantiate objects of all three subclasses and invoke methods to show dynamic dispatch.

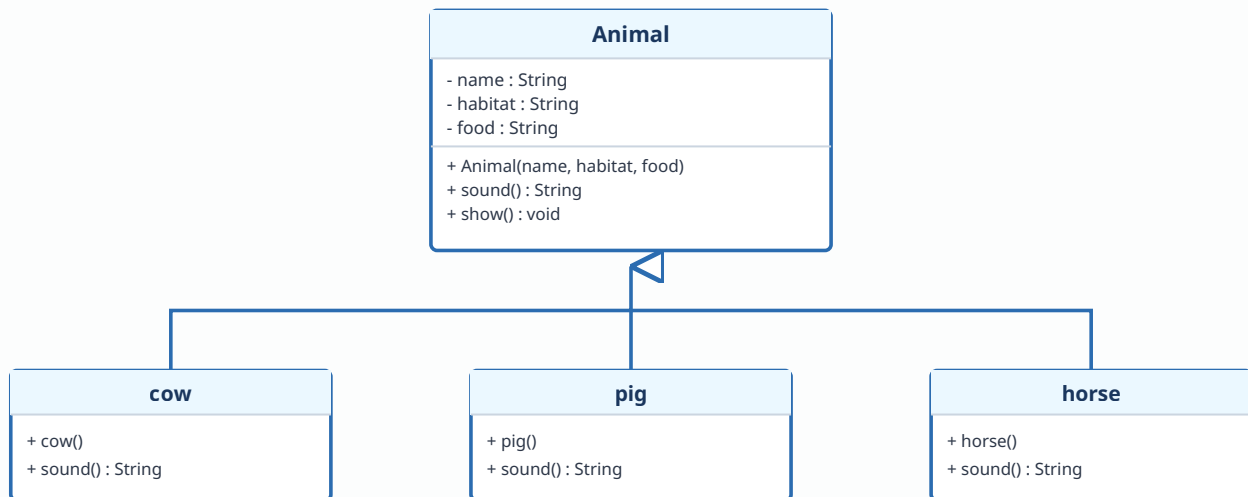
2. Step-by-Step Process

Step	Phase	Description
Step 1	Base Class Definition	Create <code>Animal</code> with encapsulated private fields: <code>name</code> , <code>habitat</code> , <code>food</code> . Initialize fields using parameterized constructor and assign via <code>this</code> .
Step 2	Encapsulation & Behaviors	Implement getter methods (<code>getName()</code> , <code>getHabitat()</code> , <code>getFood()</code>). Define <code>sound()</code> returning "animal sound" and <code>show()</code> invoking <code>sound()</code> dynamically.
Step 3	Subclass Derivation	Create classes <code>cow</code> , <code>pig</code> , and <code>horse</code> extending <code>Animal</code> . Delegate constructor values via <code>super()</code> .
Step 4	Method Overriding	Override <code>sound()</code> in <code>cow</code> ("moo"), <code>pig</code> ("oink"), and <code>horse</code> ("neigh").
Step 5	Main Execution	In <code>LabProgram3.main()</code> , instantiate <code>horse</code> , <code>pig</code> , and <code>cow</code> , and invoke <code>show()</code> .

3. Algorithm

1. **Start**
2. Instantiate `horse farm = new horse()`. Calls `super("horse", "stable", "grass")`.
3. Call `farm.show()`: prints attributes and calls overridden `sound()` ("neigh").
4. Instantiate `pig far = new pig()`. Calls `super("pig", "sty", "grains,vegetable")`.
5. Call `far.show()`: prints attributes and calls overridden `sound()` ("oink").
6. Instantiate `cow fa = new cow()`. Calls `super("cow", "shed", "grass")`.
7. Call `fa.show()`: prints attributes and calls overridden `sound()` ("moo").
8. **Stop**

4. UML Class Diagram



5. Java Source Code

```

class Animal {
    private String name;
    private String habitat;
    private String food;

    public Animal(String name, String habitat, String food) {
        this.name = name;
        this.habitat = habitat;
        this.food = food;
    }

    public String getname() { return name; }
    public String gethabitat() { return habitat; }
    public String getfood() { return food; }

    String sound() {
        return "animal sound";
    }

    void show() {
        System.out.println("animal is " + name);
        System.out.println("habitat is " + habitat);
        System.out.println("food is " + food);
        System.out.println("sound is " + sound());
        System.out.println("-----");
    }
}

class cow extends Animal {
    public cow() {
        super("cow", "shed", "grass");
    }
    @Override
    String sound() {
        return "moo";
    }
}

class pig extends Animal {
    public pig() {
        super("pig", "sty", "grains,vegetable");
    }
    @Override
    String sound() {
        return "oink";
    }
}
  
```

```

class horse extends Animal {
    public horse() {
        super("horse", "stable", "grass");
    }
    @Override
    String sound() {
        return "neigh";
    }
}

public class LabProgram3 {
    public static void main(String[] args) {
        horse farm = new horse();
        farm.sound();
        farm.show();

        pig far = new pig();
        farm.sound();
        far.show();

        cow fa = new cow();
        fa.sound();
        fa.show();
    }
}

```

6. Sample Execution Output

```

animal is horse
habitat is stable
food is grass
sound is neigh
-----
animal is pig
habitat is sty
food is grains,vegetable
sound is oink
-----
animal is cow
habitat is shed
food is grass
sound is moo
-----

```

EXPERIMENT 2: CONSTRUCTOR OVERLOADING (EMPLOYEE SYSTEM)

1. Problem Statement / Question

Write a Java program demonstrating **Constructor Overloading** and compile-time polymorphism:

- Design an Employee class containing private fields: name, id, designation, salary, and promotionstatus.
- Provide multiple overloaded constructors:
 1. **Default Constructor:** sets defaults ("not assigned", 0, "trainer", 0.0, false).
 2. **Two-Parameter Constructor:** accepts name and id.
 3. **Four-Parameter Constructor:** accepts name, id, designation, and salary.
 4. **Five-Parameter Constructor:** accepts all attributes including promotionstatus.
- Implement a show() method to display state. In driver class LabProgram5, create instances using each constructor and display employee records.

2. Step-by-Step Process

Step	Phase	Description
Step 1	Class Definition	Define class Employee with private fields: name, id, designation, salary, and promotionstatus.
Step 2	Default Constructor	Implement no-arg constructor setting default fallback values across all member variables.
Step 3	Constructor Overloading	Implement overloaded constructors accepting 2, 4, and 5 arguments respectively, printing diagnostic messages identifying which constructor was triggered.
Step 4	Display Method	Implement show() to display employee attributes in a standardized format.
Step 5	Driver Class Testing	In LabProgram5.main(), instantiate objects e1, e2, e3, and e4 invoking each constructor signature, and call show().

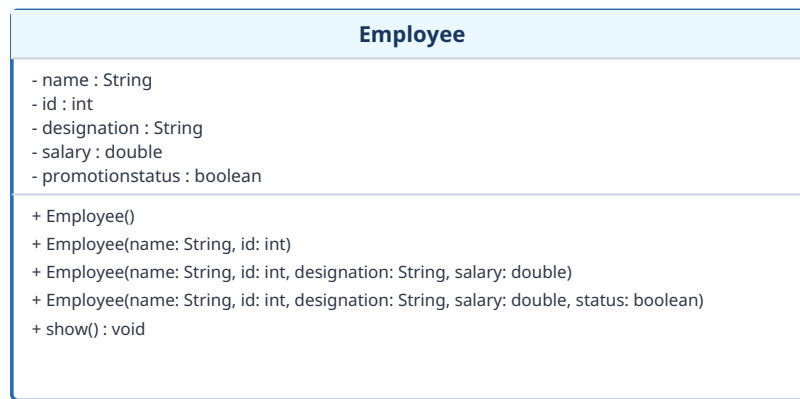
3. Algorithm

1. Start

2. Instantiate e1 = new Employee(). Invoke e1.show() to display default properties.
3. Instantiate e2 = new Employee("hemanth", 244). Invoke e2.show().
4. Instantiate e3 = new Employee("hemanth", 244, "software engineer", 70.000). Invoke e3.show().
5. Instantiate e4 = new Employee("hemanth", 244, "software engineer", 75.000, false). Invoke e4.show().

6. Stop

4. UML Class Diagram



5. Java Source Code

```

class Employee {
    private String name;
    private int id;
    private String designation;
    private double salary;
    private boolean promotionstatus;

    // Default constructor
    public Employee() {
        this.name = "not assigned";
        this.id = 0;
        this.designation = "trainer";
        this.salary = 0.0;
        this.promotionstatus = false;
    }

    // Constructor with 2 parameters
    public Employee(String name, int id) {
        this.name = name;
        this.id = id;
        this.designation = "trainer";
        this.salary = 0.0;
        this.promotionstatus = false;
        System.out.println("--- 2 parameters passing constructor");
    }

    // Constructor with 4 parameters
    public Employee(String name, int id, String designation, double salary) {
        this.name = name;
        this.id = id;
        this.designation = "trainer";
        this.salary = 0.0;
        this.promotionstatus = false;
        System.out.println("4parameters passing constructor");
    }

    // Constructor with 5 parameters
    public Employee(String name, int id, String designation, double salary, boolean promotionstatus) {
        this.name = name;
        this.id = id;
        this.designation = "trainer";
        this.salary = 0.0;
        this.promotionstatus = false;
        System.out.println("all details passing constructor");
    }

    public void show() {
        System.out.println("--- employee details ---");
        System.out.println("name : " + name);
        System.out.println("id : " + id);
        System.out.println("designation : " + designation);
        System.out.println("salary : " + salary);
    }
}

```

```

        System.out.println("promotionstatus : " + promotionstatus);
    }
}

public class LabProgram5 {
    public static void main(String[] args) {
        Employee e1 = new Employee();
        e1.show();

        Employee e2 = new Employee("hemanth", 244);
        e2.show();

        Employee e3 = new Employee("hemanth", 244, "software engineer", 70.000);
        e3.show();

        Employee e4 = new Employee("hemanth", 244, "software engineer", 75.000, false);
        e4.show();
    }
}

```

6. Sample Execution Output

```

--- employee details ---
name : not assigned
id :0
designation : trainer
salary :0.0
promotionstatus : false
--- 2 parameters passing constructor
--- employee details ---
name : hemanth
id :244
designation : trainer
salary :0.0
promotionstatus : false
4paramters passing constructor
--- employee details ---
name : hemanth
id :244
designation : trainer
salary :0.0
promotionstatus : false
all details passing constructor
--- employee details ---
name : hemanth
id :244
designation : trainer
salary :0.0
promotionstatus : false

```